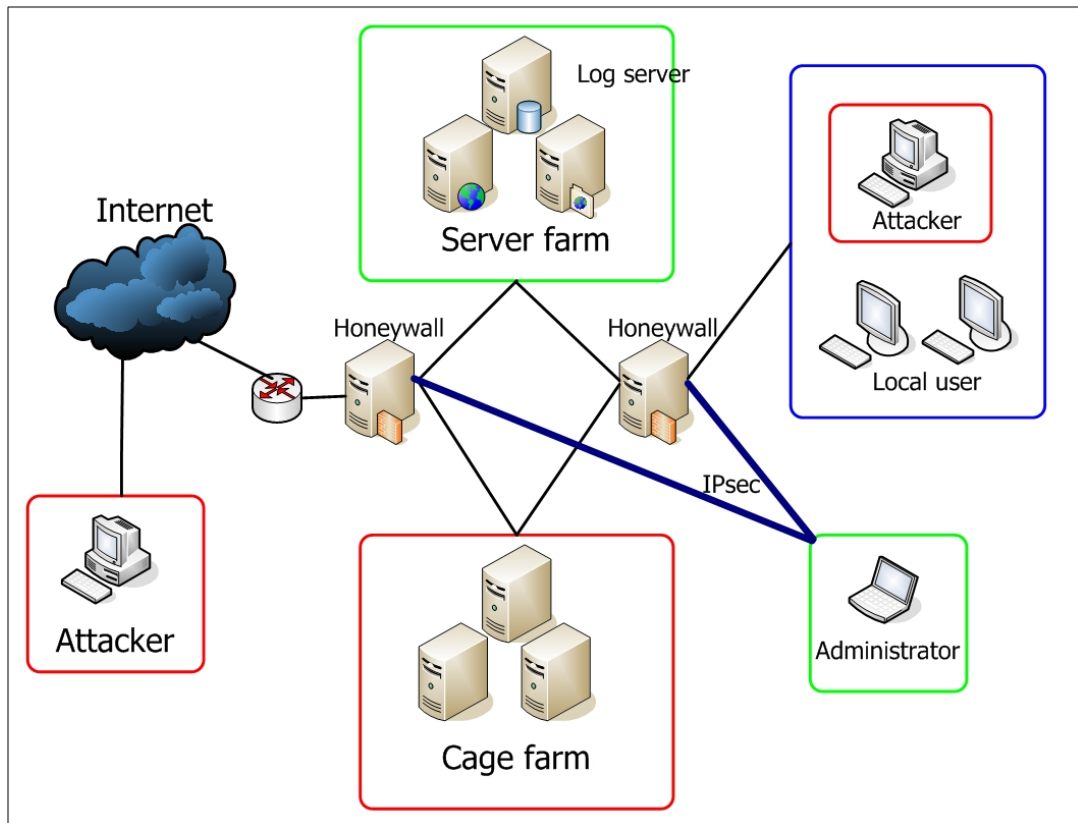


บทที่ 3

ระบบฮันนีพ็อต



รูปที่ 3.1 รูปภาพแสดงรูปภาพแสดงโครงสร้างของโปรแกรมจำแนกและบันทึกพฤติกรรมผู้บุกรุก

3.1 องค์ประกอบของระบบฮันนีพ็อต

3.1.1 Honeywall

ทำหน้าที่เป็นเครื่อง GATEWAY ที่ข้อมูลการร้องขอหรือการกระทำใดๆต้องผ่านก่อนที่จะได้รับการให้บริการ โดยจะทำงานในรูปแบบ Bridge Mode นั่นคือผู้ใช้งานจะไม่ทราบว่าการใช้งานเครื่องให้บริการได้ส่งข้อมูลผ่านตัว Honeywall แล้วจึงค่อยผ่านไปยังเครื่องให้บริการ ซึ่งการตรวจสอบและคัดแยกผู้บุกรุกออกจากผู้ใช้ปกติ หรือการดักจับโค้ดหนอน ก็จะเกิดขึ้นในตัว Honeywall นี้เอง ตัว Honeywall ได้อาศัยระบบ IDS (Intrusion Detection System) เพื่อให้มีความสามารถที่ได้กล่าวมาแล้ว และเมื่อตรวจพบว่าการกระทำเข้าข่ายการบุกรุก ตัว Honeywall จะทำ

การเปลี่ยนให้ผู้บุกรุกไปใช้งานเครื่องกับดักแทน ซึ่งได้เตรียมรอไว้แล้ว และมีสภาพแวดล้อมและค่าต่างๆเหมือนเครื่องให้บริการจริง และตัว Honeywall ก็จะส่งข้อมูลไปให้กับเครื่อง Log server

และที่ตัว Honeywall นี้เองจะเป็นตัวควบคุมระบบทั้งหมด โดยผ่านทางโปรแกรมที่ผู้พัฒนาได้จัดทำขึ้น นั่นคือโปรแกรมTM (Tartarus management) ซึ่งโปรแกรมนี้สามารถควบคุมจัดการและเรียกข้อมูลทุกอย่างได้ โดยเรียกจากโปรแกรมที่ผู้พัฒนาได้สร้างขึ้นและฝังการทำงานไว้ในส่วนต่างๆ นอกจากนี้จะสามารถเรียกใช้โปรแกรมควบคุมและจัดการนี้ได้โดยตรงจากเครื่องHoneywall แล้ว ทางผู้พัฒนายังได้จัดเตรียมช่องทางที่ผู้ดูแลระบบจะสามารถเข้ามาบริหารจัดการได้ผ่านทางช่องทางที่ปลอดภัยโดยใช้ IPsec (IPsecurity) ซึ่งจะมีการAuthentication และ Encryption ทำให้สามารถจัดการระบบผ่านเครือข่ายได้ด้วย

3.1.2 เครื่องกับดัก (Cage)

ทำหน้าที่จำลองตัวเองให้เหมือนเครื่องให้บริการมากที่สุด และอาจมีการเปิดบริการเสริมหลอกล่อไว้ด้วย เพื่อให้ผู้บุกรุกหลงเชื่อและแสดงพฤติกรรมในการบุกรุกต่างๆ โดยการกระทำต่างๆ ไม่ว่าจะเป็นการกดเป็นพิมพ์ส่งคำสั่งเข้ามา การดาวน์โหลดหรือการนำข้อมูลเข้ามาในตัวเครื่องกับดัก หรือการนำข้อมูลออก จะถูกบันทึกพฤติกรรมไว้หมด ไม่ว่าจะเป็นส่งคำสั่งนี้จะใช้ผ่านทางช่องทางที่มีการเข้ารหัสก็ตาม เพราะเครื่องมือที่ใช้บันทึกพฤติกรรมทำงานในรูปแบบของ Kernel-base และสามารถซ่อนตัวเองไม่ให้ถูกพบที่กำลังทำงานอยู่ นอกจากนี้เครื่องกับดักจำเป็นที่จะต้องมีการรักษาความปลอดภัยในด้าน การถูกนำไปใช้เป็นฐานในการโจมตีเครื่องอื่นๆ โดยได้มีการตรวจสอบสภาพและความเสียหายอยู่ตลอดเวลา หากถึงจุดที่เครื่องกับดักมีความอ่อนแอถึงจุดที่กำหนด (ซึ่งผู้ดูแลระบบสามารถกำหนดได้ตามความสมควร) เครื่องกับดักก็จะถูกระงับการใช้งานโดยอัตโนมัติ

3.1.3 เครื่องแม่ข่ายฐานข้อมูล (Log server)

ทำหน้าที่เก็บบันทึกรายละเอียดที่ส่งเข้ามาจากที่ต่างๆ ลงในฐานข้อมูลอย่างมีระบบ ซึ่งข้อมูลที่ส่งมา จะมาจากทุกส่วน เช่น Honeywall, Cage ซึ่งเครื่องแม่ข่ายฐานข้อมูลนี้จะอยู่ในส่วนที่ปลอดภัยจากการบุกรุกเข้ามา คืออยู่ภายใน DMZ ของระบบ ข้อมูลที่ถูกจัดเก็บในเครื่องนี้ก็ได้แก่ การกระทำของผู้บุกรุกต่างๆ, ไฟล์ที่ผู้บุกรุกได้นำเข้าหรือเอาออกจากระบบ, ข้อมูลการเปลี่ยนแปลงและสภาพของเครื่องกับดัก, ไฟล์อิมเมจของเครื่องกับดักที่ถูกกระงับการใช้งาน เป็นต้น

3.2 หน้าที่การทำงานในแต่ละส่วน และเครื่องมือที่ใช้

การทำงานของโปรแกรมจะแบ่งออกเป็น 5 ส่วนใหญ่ๆ ดังนี้

- จำแนกผู้บุกรุกออกจากผู้ใช้งานทั่วไปและส่งเข้าใช้งานเครื่องกับดัก
- ส่วนเฝ้าบันทึกพฤติกรรม และควบคุมขอบเขตของผู้บุกรุก
- ส่วนจัดเก็บข้อมูลกลาง
- ส่วนการจัดการและตรวจสอบสภาพความเสียหายของเครื่องกับดัก
- ส่วนการควบคุมและแสดงผลการบันทึกข้อมูลต่างๆของระบบ

3.2.1 จำแนกผู้บุกรุกออกจากผู้ใช้งานทั่วไปและส่งเข้าใช้งานเครื่องกับดัก

ในส่วนนี้จะรับผิดชอบโดยตัวHoneywall โดยใช้การจัดจำแนกแบบ Rules-base

3.2.1.1 การจำแนกเป็นผู้บุกรุก

ในการจำแนกผู้บุกรุกนั้นจะพิจารณาว่าผู้ใดเป็นผู้บุกรุกหรือเข้าข่ายเป็นผู้บุกรุก โดยใช้กฎของโปรแกรมตรวจจับผู้บุกรุก (Snort_inline) ซึ่งเป็นเครื่องมือประเภท IDS (Intrusion Detection System) โดยหากว่าการเชื่อมต่อใดมีลักษณะตรงตามกฎของโปรแกรมตรวจจับผู้บุกรุกจะถือว่าเจ้าของการเชื่อมต่อที่เข้าข่ายเป็นผู้บุกรุกจะต้องทำการเปลี่ยนทิศทางการเชื่อมต่อดังกล่าวไปยังเครื่องกับดักเพื่อดำเนินการบันทึกพฤติกรรมของผู้บุกรุกนั้นและเพื่อให้การจำแนกผู้บุกรุกของโปรแกรมตรวจจับผู้บุกรุกเป็นไปตามความต้องการของผู้พัฒนามากขึ้น ทางผู้พัฒนาจึงได้ทำการแก้ไขกฎการจำแนกบางส่วน เช่น Mysql cazz exploit

กฎของการตรวจสอบการบุกรุกของเครื่องมือที่ใช้เข้าโจมตีโปรแกรมฐานข้อมูล ที่ชื่อว่า มายเอสคิวแอล (Mysql) โดยชื่อโปรแกรมที่เรียกว่า cazz เพื่อเข้าโจมตีแบบยัดครองระบบ เพื่อตั้งให้กฎเองมีความยืดหยุ่นที่ว่าหากเข้ามาเชื่อมต่อเข้ามายังช่องทางหมายเลข สามสามศูนย์หก ลักษณะการขอการสถาปนา ก็ให้ถือว่าเป็นการโจมตีจากผู้บุกรุก ซึ่งมีตัวอย่างดังนี้

```
alert tcp $EXTERNAL_NET any -> $SQL_SERVERS 3306
(msg:"MYSQL root login attempt"; flow:to_server,established;)
```

3.2.1.2 การจำแนกเป็นผู้ใช้ปกติ

ในการจำแนกผู้ใช้นั้นจะพิจารณาโดยใช้กฎของโปรแกรมตรวจจับผู้บุกรุก

เช่นเดียวกับการจำแนกผู้บุกรุก คือการเชื่อมต่อใดที่ไม่ตรงกับกฎให้ถือว่าเจ้าของการเชื่อมต่อนั้นๆ เป็นผู้ใช้ปกติสามารถให้ผ่านไปใช้บริการของเครื่องแม่ข่ายจริงได้ตามปกติ และกรณีที่ไม่ต้องการให้มีการตรวจสอบเครื่องบางเครื่องที่เชื่อมต่อเข้ามา สามารถทำการกำหนดกฎที่ ไฟร์วอลล์ให้การเชื่อมต่อ นั้นสามารถผ่านเข้าไปได้โดยไม่ต้องมีการตรวจสอบ เนื่องจากว่าผู้ที่ควบคุมการเชื่อมต่อจริงแล้วคือ ไฟร์วอลล์ ในการตรวจสอบของโปรแกรมตรวจจับผู้บุกรุกอาจมีผิดพลาดได้จึงจำเป็นต้องตั้งกฎของระบบตรวจจับผู้บุกรุกให้มีความรัดกุมและบ่งบอกชัดเจนว่าผู้ใดเป็นผู้บุกรุก เพื่อให้กระบวนการตรวจสอบนั้นไม่เกิดผลการเข้าใจผิดว่า ผู้ใช้งานปกตินั้นเป็นผู้บุกรุก

3.2.1.3 การจำแนกว่าเป็นโค้ดหนอน

ในการระบุได้ว่าเป็นโค้ดหนอน จะใช้การพิจารณาแบบ Rules-base เช่นเดียวกับการตรวจจับผู้บุกรุก โดยจะใช้ฐานข้อมูลที่เก็บSignatureของโค้ดหนอนชนิดต่างๆเอาไว้ มาเปรียบเทียบกับรูปแบบที่พบ ซึ่งจำเป็นจะต้องมีการปรับปรุงฐานข้อมูลSignatureนี้ให้ทันสมัยอยู่ตลอดเวลา ซึ่งทางผู้พัฒนาได้เพิ่มเข้าไปในส่วนของ Snort_inline เพื่อให้สามารถตรวจสอบโค้ดหนอนไปพร้อมกับตรวจจับผู้บุกรุก

3.2.1.4 เครื่องมือที่ใช้ในส่วนของ Honeywall

ทางผู้พัฒนาได้ประยุกต์เครื่องมือต่างๆให้สามารถใช้งานและติดต่อส่งผ่านคำสั่งถึงกันได้ ทำให้ได้ความสามารถตามที่ต้องการ

Iptables

ไอพีเทเบิลนั้นเป็นโปรแกรมไฟร์วอลล์แบบState Full Inspector ที่ทำงานใน Kernel-Mode โดย iptables จะมีหน้าที่ในการจัดการเชื่อมต่อ และการอนุญาตหรือไม่อนุญาตให้ขึ้นข้อมูลต่างๆ สามารถผ่านเข้าออกระบบเครือข่ายได้ ซึ่งในโครงการนี้ได้นำโปรแกรม iptables มาใช้งานเพื่อให้ขึ้นข้อมูลที่ผ่านเข้ามาในระบบเครือข่ายถูกส่งไปให้ส่วนที่โปรแกรมตรวจจับผู้บุกรุกที่ทำงานในยูสเซอร์โหมดนำขึ้นข้อมูลเหล่านั้นไปพิจารณาต่อไป โดยขึ้นข้อมูลจะถูกส่งผ่านไปทางโมดูลที่ชื่อว่า ip_queue และอีกหน้าที่หนึ่งของ iptables คือการจัดเส้นทางให้การเชื่อมต่อให้กับผู้ใช้ที่ถูกจำแนกแล้วว่าเป็นผู้ใช้ปกติหรือเป็นผู้บุกรุกระบบ ให้เข้าไปใช้งานเครื่องให้บริการจริง หรือเครื่องกับดัก โดยตัวไอพีเทเบิลจำเป็นจะต้องทำงานร่วมกับ Snort_inline โดยผ่านการเชื่อมโยงให้สามารถทำงานประสานกันได้โดย S2I

snort_inline

สนอร์ตอินไลน์เป็นโปรแกรมไอดีเอสที่ทำงานในยูสเซอร์โหมด ซึ่งจะทำหน้าที่เป็นระบบตรวจจับผู้บุกรุก ในโครงงานนี้จะนำ snort_inline มาใช้ในการจำแนกว่าผู้ใดเป็นผู้ใช้งานปกติและผู้ใดเป็นผู้บุกรุกระบบ ซึ่งจะจัดจำแนกตามข้อมูลหรือกฎที่ผู้ดูแลระบบใช้ ซึ่งในที่นี้โปรแกรมจะทำการอ่านค่าขึ้นข้อมูลที่ส่งมาทางมอดูล ip_queue แล้วนำมาเปรียบเทียบกับกฎที่ได้ตั้งไว้ ว่าเข้าข่ายเป็นการโจมตีหรือไม่ และทำการแจ้งเตือนในกรณีที่มีการตรวจสอบมีผลออกมาว่าขึ้นข้อมูลที่ส่งมามีลักษณะของการโจมตีระบบ

ซึ่งตัวสนอร์ตอินไลน์สามารถนำมาประยุกต์เพิ่มกฎที่เหมาะสมให้สามารถดักจับโค้ดหนอนที่แพร่ระบาดเข้ามายังระบบได้ โดยเพิ่มในส่วนของ Signature เข้าไปเพื่อใช้เป็นฐานข้อมูลให้ตัวโปรแกรมสามารถรู้จักและสามารถแยกแยะว่าเป็นโค้ดหนอนได้

S2I

เอสทูไอ (S2I: snort command to iptables) เป็นโปรแกรมที่สร้างมาจากภาษาเพิร์ล เพื่อให้สามารถแปลงการแจ้งเตือนที่ได้มาจากการไปคิวรีกับฐานข้อมูล ไปเป็นคำสั่งของโปรแกรม iptables เพื่อโปรแกรมไฟร์วอลล์จัดเส้นทางให้กับขึ้นข้อมูลที่มาจากผู้ใช้งานปกติผ่านไปยังส่วนที่เป็นเซิร์ฟเวอร์จริง และในส่วนของผู้บุกรุกเองนั้นก็ให้ส่งไปยังเครื่องกับดักที่ได้เตรียมไว้ นั่นก็คือเป็นตัวกลางที่จะทำให้โปรแกรม Snort_inline สามารถทำงานประสานกับโปรแกรมIptablesได้ ซึ่งสาเหตุที่ต้องมีเอสทูไอนั้นเนื่องมาจากโปรแกรมSnort_inline และโปรแกรมIptables ที่ใช้ในโครงงานนี้นั้นทำงานในโหมดที่ต่างกันจึงติดต่อกันเองโดยตรงไม่ได้ ดังนั้นจึงต้องมีโปรแกรมสื่อกลางช่วยในการติดต่อระหว่างกัน

TM (Tartarus Management)

Tartarus Management เป็นโปรแกรมในรูปแบบ GUI ใช้งานง่ายไม่จำเป็นต้องจดจำคำสั่งของแต่ละเครื่องมือ ซึ่งมีเป็นจำนวนมากในระบบ โดยเป็นโปรแกรมที่ใช้ควบคุมการทำงานทั้งหมดของ Honeypot Program Suite ซึ่งได้ช่วยแก้ปัญหาความยุ่งยากในการใช้งานระบบอันนี้พอดีที่ผ่านมาได้ เพราะได้มีการรวมการบริหารทั้งหมด ทั้งในส่วนของการดูแลและจัดการเครื่องมือ การตั้งค่าการใช้งานต่างๆ เช่นค่าการใช้งานของ Snort_inline ทั้งในส่วนของserverและclient, ค่าการใช้งาน sebekทั้งในส่วนของserverและclient, ค่าการใช้งานของ samhainทั้งในส่วนของserverและclient,

รวมทั้งการจัดการเครื่องที่ทำหน้าที่เป็นCage ทั้งการตรวจเช็คสภาพและการปรับเปลี่ยน ทุกอย่างมาไว้ในโปรแกรมควบคุมจัดการนี้

นอกจากนี้ยังใช้ในการเรียกแสดงข้อมูลทุกอย่างที่มีภายในระบบ เช่น ข้อมูลที่เก็บพฤติกรรมผู้บุกรุก หรือข้อมูลสภาพความเปลี่ยนแปลงของเครื่องกับดัก เป็นต้น

3.2.2 ส่วนเฝ้าบันทึกพฤติกรรม และควบคุมขอบเขตของผู้บุกรุก

ในส่วนนี้มีชื่อเรียกว่า “Cage” จะเป็นส่วนของกับดักหรือระบบหลอกที่ใช้เพื่อให้ผู้บุกรุกเข้ามาติดกับและเฝ้าคอยดูพฤติกรรมของผู้บุกรุกที่อยู่ในเครื่องกับดัก และมีการใช้การควบคุมข้อมูล Data control เพื่อความปลอดภัย นอกจากนี้ยังได้พัฒนาด้านความแนบเนียนของเครื่องกับดักให้มีสภาพแวดล้อมและค่าต่างๆ ให้เหมือนกับเครื่องที่ใช้งานจริง และมีความปลอดภัยไม่ให้เครื่องกับดักถูกใช้เป็นฐานในการโจมตีเครื่องอื่นๆ

Data control

Data control จะป้องกันเหล่าผู้โจมตีจะใช้ตัว honeypot เพื่อโจมตีหรือ ทำอันตรายในเครื่องอื่นๆที่ไม่ใช่ระบบHoneyPot ตัว Data control จะลดอัตราเสี่ยง โดยมันทำการจำกัด จำนวนข้อมูลที่ส่งออก โดยใช้ NIPS(Network Intrusion Prevention System) ต่อเข้ากับตัวนับเพื่อนับว่าหากถึง limit ที่เข้ามาจำนวนมาก honeypot สามารถกำหนดได้ NIPSจะสามารถ block รูปแบบการกระทำได้ รวมทั้ง2 อย่างนี้เข้าไว้ด้วยกันจะทำให้เกิดความยืดหยุ่นที่ data control โดยติดตั้งไว้ที่ เครื่อง Honeywall เพื่อที่จะได้ควบคุมทั้งขาเข้าและขาออก โดยเริ่มแรกผู้ดูแลระบบจะต้องตัดสินใจว่าจะยอมรับให้มีการติดต่อเข้ามาอย่างน้อยเพียงใดต่อช่วงเวลา โดยจะนับการ connect ที่มาจากภายนอก และจะให้ limit เท่าใดเพื่อป้องกันการ connect ที่จะมีมากมายเป็นการลดอัตราเสี่ยงของการ scan การโจมตีหรือการ denial of service ซึ่งยากที่จะป้องกันไม่ให้เกิดโจมตีจำนวนมาก แต่การจำกัดจำนวนของแพ็คเกจก็มีข้อเสียตรงที่ผู้บุกรุกอาจ สังเกตได้ว่า ถ้าหากพวกเขาถูก block หลังจากทำการส่ง packet เข้ามาเป็นจำนวนหนึ่ง เช่นจะถูก block เมื่อส่งpacket ที่ 11 ก็อาจทำให้ผู้บุกรุกทราบได้ว่ามีการทำ Data control เอาไว้ ค่าพื้นฐานสำหรับการเชื่อมต่อจะกำหนดได้ใน re.firewall ตัวค่าvariable อื่นๆ ที่เป็น IP Protocol มีNOT,TCP,UDP หรือ ICMP (เช่น IPsec,IPv6,tunneling,Network Voice Protocol และอื่นๆ) ตัวอย่างการกำหนดจำนวนpacket ที่เข้าออกเป็นดังนี้

Set the connection outbound limits for different protocols

SCALE="day"

TCPRATE="15"

UDPRATE="20"

ICMPRATE="50"

OTHERRATE="15"

มีการสนับสนุนจาก limit กับ rc.firewall ซึ่งขั้นต่อไปคือการทำให้สนับสนุน NIPS ซึ่งเป็น function ที่ว่าเป้าหมายของ NIPSคือการจำแนก และ block เมื่อรู้ว่าเป็นการโจมตี มันทำหน้าที่นี้โดยจะสอดคล้องดู Packet อื่นๆที่ผ่านเข้ามายัง Honeywall ถ้าตัว packet นั้น match กับ IDS rules มันจะไม่เพียงแค่สร้างสัญญาณเตือน(เหมือน NIDS) แต่ packet จะถูก drop (ป้องกันการโจมตี) หรือเปลี่ยนแปลง(หยุดการโจมตี) จะมีประโยชน์ที่จะลดอัตราเสี่ยงของการโจมตีขาออกได้

Iptables script can be used with the snort_inline filter

#QUEUE="no"

#Do not use experimental QUEUE support

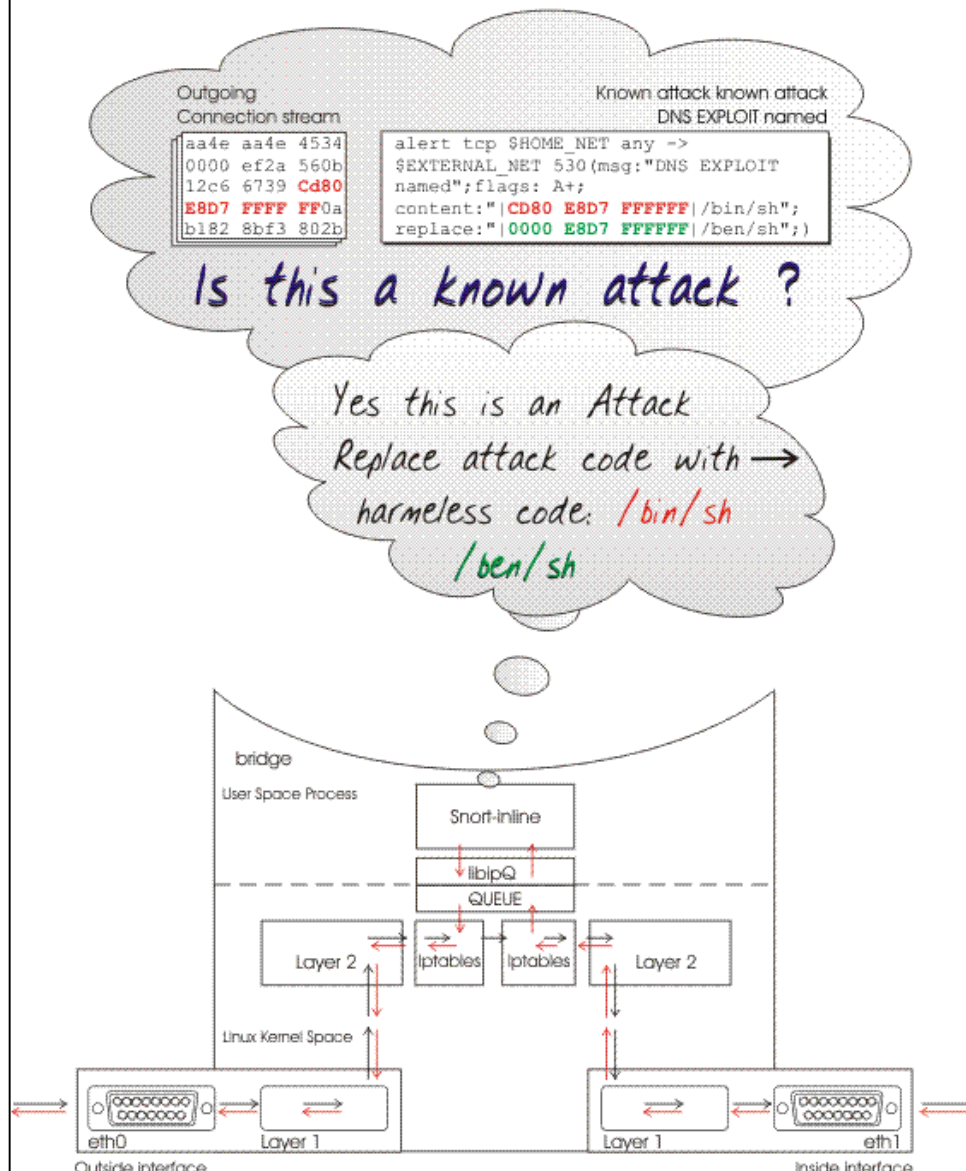
QUEUE="yes"

#Use experimental QUEUE support

เราได้ตั้งคำให้ยินยอม ให้เครื่องกับดักส่งข้อมูลออกได้ หรือทำการส่ง ICMP, Ping เพื่อเฝ้าดูพฤติกรรม เนื่องจากหากทำการใช้กฎที่มีทั้งหมด จะทำให้ผู้บุกรุกไม่สามารถทำอะไรได้เลย การใช้กฎของ snort เพื่อเป็นการป้องกันการโจมตีนั้น ในองค์กรต่างๆ อาจมีความต่างของการกำหนดว่าอะไรคือการโจมตี ดังนั้นพวกทางผู้พัฒนามุ่งเน้น ด้านการดัดแปลงตัวกฎของ snort_inline ให้ทำงานได้ตามที่ต้องการ ก่อนที่จะนำมากำหนดใช้ ดังนั้นกฎที่เราใช้จะมีส่วนที่กลับกันจากรูปแบบที่เป็นกฎทั่วไปของตัว snort_inline ที่เป็นกฎพื้นฐานโดยที่จะจับตาที่การโจมตีขาเข้า แต่ทางระบบฮันนีพ็อต จะเพิ่มการจับตาดูที่การโจมตีขาออก เป้าหมายของการป้องกันจากโลกภายนอกจากระบบฮันนีพ็อต

จะตั้งกฎเพื่อให้วัตถุประสงค์ที่เป็นส่วนของการวิเคราะห์ตัว packet และ ถ้าหากเห็นว่าเป็นการโจมตีจากภายนอก การ block หรือ drop ตัว packet จะบรรจุการโจมตีไว้ ทำการ block หรือ drop ตัว packet ดูรูป honeypot3 ที่บรรจุการโจมตีจะสามารถเห็นรูปแบบนี้ได้จากตัวอย่างของการ drop ตัว Code Red II ซึ่งเป็นการโจมตีที่เปรียบเทียบให้เห็นง่ายๆ โดยการใช้วิธีการวางทับ replace ruleset จะไม่ทำการ block ดังรูปแบบการdrop ของระบบตรวจจับผู้บุกรุก

REPLACE MODE OPERATION



รูปที่ 3.2 รูปแบบการวางทัพบข้อมูลในแพ็คเกจของระบบตรวจจับผู้บุกรุก

การแทนที่ในส่วนนี้คือการดัดแปลงตัว Contents ต่างๆ ที่เป็นการโจมตีในปัจจุบัน หยุคยัง การเข้าใช้ระบบ exploit นี้เป็นส่วนแยกของความต่างที่เป็นการควบคุมการกระทำสำหรับตรวจจับ attacker พวกเขาจะเห็น Attacker ต่างๆ นั้นไปถึงในเป้าหมายที่ได้เจตนาที่จะไปไว้ แต่ก็ไม่สามารถ แสดงออกมาได้ว่าทำไม Attacker จึงพลาดในการกระทำนั้น ตัวเครื่องมือ snortconfig เป็นตัว script ที่ ช่วยทำให้เกิดความยืดหยุ่นนี้กลับกันกับจากกฎพื้นฐานของตัว snort_inline ที่สามารถทำได้ เช่น การ drop การ sdrop หรือ การ replace โดยที่เครื่องมือนี้มีความสำคัญเป็นอันมากรักษากฎของกฎที่ได้ จัดเตรียมไว้ของ snort_inline

สำหรับวัตถุประสงค์ของ แปลงของทางระบบอันนี้เพื่อจะติดตั้งเป็นแบบ Drop ที่การจัดตั้ง กฎ และ ในส่วน snort_inline.conf โดยการเข้าไป configure file ที่ /etc/snort_inline ซึ่งได้ใช้การ แบ่งตัวเพิ่มข้อมูลจาก /etc/snort ดังนั้นจะไม่ทำให้สับสน ในการเริ่มต้นกับตัว snort_inline โดยจะใช้ snort_inline startup script ตัว script นี้เป็นตัวเริ่มต้นของการจัดค่าตัวแปรให้เหมาะสมกับ snort_inline ที่จะสามารถทำงานได้ จะช่วยให้การใช้งานง่ายขึ้น ซึ่งจะถูกรวมไว้ในชุดโปรแกรมจำแนกและบันทึก พฤติกรรมผู้บุกรุก ในส่วนของโปรแกรม TM ไว้หมดแล้ว

Sebek_client

เซเบคไคลเอนจะคอยซ่อนตัวอยู่ในกับดักเพื่อเฝ้าดูพฤติกรรมและการกระทำของผู้บุกรุก และ คอยส่งข้อมูลนั้นไปยังเครื่องที่ใช้เก็บพฤติกรรมลงฐานข้อมูล โดยส่งผ่านช่องทางที่มีความปลอดภัย มี การเข้ารหัสข้อมูลในรูปแบบของ เซเบคเอง

เซเบคเป็นData Capture Toolแบบ Kernel-based โดยในการนำมาใช้จะประยุกต์ใช้เป็นแบบ LKM (Loaded Kernel Module) มีความโดดเด่นในความสามารถในการดักจับข้อมูลได้ทุกการกระทำ ของIntruderที่กระทำผ่าน read() system call ไม่ว่าจะเป็นในส่วนของ keystroke, files transfer, burneye passwords หรือหากมีการใช้ IRC clientหรือ e-mail client ตัวเซเบคก็สามารถดักจับได้ ดังเช่นเมื่อIntruderมีการนำfileส่งเข้ามา ตัวเซเบคก็สามารถอ่านและบันทึกไฟล์นั้นไว้ได้ ทั้งยังสามารถใช้เป็นGlass-box เมื่อเปรียบเทียบกับการทำงานของblack-boxที่เรารู้จัก นั่นคือเรา สามารถใช้ติดตามการทำงานของโปรแกรมที่intruderสามารถรันได้ด้วย แม้ว่าตัวintruder จะlog out ออกไปแล้ว

เซเบคมีรูปแบบวิธีการปิดซ่อนตัวเองได้อย่างมีประสิทธิภาพ และค่อนข้างยากที่จะตรวจสอบ พบทั้งจากภายในและภายนอก โดยจะทำการปิดซ่อนโมดูลที่ใช้ และลบlinked list ที่จะแสดงการมี ตัวตนออก โดยใช้ the cleaner ทำให้Intruder ไม่สามารถตรวจสอบพบว่าเซเบคกำลังทำงานอยู่ภายใน เครื่อง ทุกpacketของเซเบค ตัวเซเบคจะมีfunctionในการสร้างและส่งเอง โดยไม่ใช่ TCP/IP stack

ดังนั้นจึงไม่สามารถถูกมองเห็นและไม่สามารถ block traffic ของเซเบคได้ หลังจากสร้าง packet แล้ว packet จะถูกส่งไปยัง device driver โดยตรง ไม่ผ่าน raw socket socket interface ซึ่งพวกโปรแกรม sniffer ที่ใช้ libpcap เป็นฐานจะไม่สามารถเห็นได้เพราะมันไปดักจับข้อมูลที่ raw socket interface และเพื่อป้องกันการตรวจจับในเครือข่าย sebek จะไม่ใช่ ARP แต่จะกำหนด MAC Address ของ server ไว้เลย เพื่อไม่ต้องทำการ ARP request เพื่อป้องกันการทำ ARP spoofing ซึ่งก็จะลดการดักจับข้อมูลในเครือข่ายได้ ทั้งนี้เซเบคมีรูปแบบการส่งข้อมูลด้วยโปรโตคอล UDP

TMdaemon

TMdaemon เป็นเซอร์วิสที่ผู้พัฒนาได้สร้างขึ้นเพื่อคอยตรวจสอบสถานะของเครื่องกับดักและจะทำการหยุดการทำงานของเครื่องกับดักโดยอัตโนมัติ หากเครื่องกับดักมีความอ่อนแอเกินกว่าที่จะสามารถควบคุมพฤติกรรมของผู้บุกรุกเอาไว้ได้ โดยการตรวจสอบข้อมูลดังกล่าวจะกระทำโดยใช้โปรแกรมที่เขียนขึ้นมาให้ไปอ่านค่าจากฐานข้อมูลต่างๆ เพื่อมาพิจารณาว่าตรงตามเงื่อนไขที่กำหนดไว้หรือไม่ ถ้าตรงก็จะทำการหยุดการทำงาน (suspend) ซึ่งจุดที่จะบ่งบอกว่าเครื่องกับดักจะถูกหยุดการทำงานผู้ดูแลระบบสามารถตั้งค่าได้ตามความเหมาะสม โดยเรียกใช้งานผ่านโปรแกรมควบคุม Tartarus Management (TM)

portsentry

พอร์ตเซนทรีจะเป็นตัวที่ทำหน้าที่ในการจำลองบริการที่ระบบจริงไม่ได้เปิดให้บริการอยู่ เพื่อเป็นการหลอกล่อผู้บุกรุกในกรณีที่ผู้บุกรุกร้องขอการเชื่อมต่อนั้นๆ เข้ามายังระบบ ซึ่งจะทำให้ผู้บุกรุกเชื่อว่ามีบริการเปิดบริการดังกล่าวอยู่ และเริ่มทำการโจมตี จากนั้นอันนี้พอร์ทก็จะทำการบันทึกพฤติกรรมต่างๆ ของผู้บุกรุกไว้

Samhain (samhain)

เซมเฮนเป็นเซอร์วิสที่ทำหน้าที่ในการตรวจสอบความถูกต้องของไฟล์สำคัญๆ ในระบบว่าผู้บุกรุกได้ทำการเปลี่ยนแปลงอะไรไปบ้าง เช่นไฟล์ไคถูกแก้ไข ไฟล์ไคโดนลบ โดยโปรแกรมจะทำการบันทึกการเปลี่ยนแปลงของไฟล์ต่างๆ และส่งไปเก็บลงในฐานข้อมูลในเครื่อง Log server ซึ่งทำให้เราทราบความเปลี่ยนแปลงของระบบได้ อีกทั้งตัวโปรแกรมนี้อาจสามารถทำการซ่อนตัวจากการตรวจสอบจากผู้บุกรุกได้ด้วยในระดับหนึ่ง

3.2.3 ส่วนจัดเก็บข้อมูลกลาง (Log server)

จะทำการเก็บรวบรวมข้อมูล จัดเก็บลงฐานข้อมูล รับผิดชอบโดยเครื่องแม่ข่ายฐานข้อมูล (Log server) ซึ่งสิ่งที่จะต้องพิจารณาในการจัดตั้ง Log server คือ ความปลอดภัยของตัวเครื่อง Log server เอง และรูปแบบการส่งข้อมูลเข้ามา เนื่องจากหากรูปแบบที่ใช้ในการส่งข้อมูลเข้ามาเป็นการส่งข้อมูลแบบธรรมดา ผู้บุกรุกอาจใช้เครื่องมือช่วยในการดักจับ และทราบถึงการมีอยู่ของระบบอันนี้เพื่อค และ อาจก่อให้เกิดปัญหาด้านความปลอดภัยของระบบจริง

ข้อมูลสำคัญที่จะทำการเก็บลงฐานข้อมูล เพื่อสามารถนำไปวิเคราะห์

1. ข้อมูลเบื้องต้นของผู้บุกรุก เช่น หมายเลขไอพีแอดเดรส
2. ข้อมูลการป้อนคำสั่ง หรือ keystroke ของผู้บุกรุก
3. ข้อมูลที่นำเข้า หรือ อาจเป็นไฟล์หรือเครื่องมือที่ผู้บุกรุกจะนำมาใช้
4. ข้อมูลที่ส่งออกจากเครื่องกับดัก อาจเป็นข้อมูลหรือเป็นการกระทำเพื่อโจมตีระบบ

ตัว file logs ของ Firewall จะมีทุกๆ ข้อมูลพื้นฐานและพร้อมแล้วที่จะทำในส่วนนี้โดยการนำ rc.firewall script เข้ามาใช้โดยระบบจะพร้อมที่จะจับตาการเชื่อมต่อ ทั้งขาเข้าและขาออก ไปสู่ file /var/log/message นี่เองจะเป็นข้อมูลที่สำคัญ เพราะว่าจะเป็นเครื่องบ่งชี้ว่า Attacker ทำอะไรอยู่ มันจะเตือนครั้งแรกเลยเมื่อเริ่มมีการเชื่อมต่อขาออก หรือ การโจมตีขาออกโดยพื้นฐานการทำงานนี้ ตัว firewall จะตรวจสอบความล่อแหลมอย่างรวดเร็วในความต่างที่เป็นพฤติกรรมใหม่ หรือ พฤติกรรมที่ไม่รู้นั่นเอง ตัว script เองจะมี สิ่ง ความแตกต่างของการเดิน packet ดังนี้ TCP, UDP, ICMP และ OTHER จะมีตัว Data Control ตัว OTHER จะเข้ามาแทนทุกๆ อันที่เป็น non-IP proto 1, 6 หรือ 17 ชนิดเหล่านี้ มันจะมีความสนใจต่อเมื่อมีบางคนใช้ non-standard IP traffic เหล่านี้เหมือนกับเป็นการพยายามโจมตีใหม่ๆ หรือ วิธีการที่ไม่เคยพบมาก่อน อาจจะเป็นช่องทางของ Backdoor ที่เคยมีการค้นพบ อ่านได้ในบทความ Scan of the month 22 (<http://www.honeynet.org/scans/scan22/>)

ในส่วนที่สองเป็นการตรวจจับทุกๆ Packet และ มันมีประโยชน์มากในการจับทั้งเมื่อเข้ามา และ เมื่อออกไปจากตัวอันนี้เพื่อค โดยใช้มาตรฐานของ snort.conf configuration file นี่จะเป็นการ configure file เพื่อตรวจจับทุกๆ IP ที่สัญจรผ่านในเครือข่าย ไปสู่ tcpdump log file สำหรับนำมาวิเคราะห์ต่อในภายหลัง ดังนั้น การจะสั่งให้มีการเก็บข้อมูลจาก snort ตามที่ต้องการ อย่างอัตโนมัติจะกระทำผ่าน snort.sh start script จะเป็นตัวเริ่มการทำงานเพื่อให้ไม่มี cron ในทุกวัน แจ้งให้ทราบว่า จะทำอะไรในการใช้ startup script ทางผู้พัฒนาได้สร้างการตรวจจับไว้ที่ส่วน interface

ในส่วนที่สามนี้เป็นการทำทนายการดักจับเหล่าผู้โจมตีบนตัว Honeypot เองมันเป็นแบบง่าย มันมีการใช้กันมานานแล้วกับระบบที่เป็นแบบ cleartext protocol เช่น FTP, HTTP และ Telnet คุณแค่เพียงมีตัว sniff ที่ดักจับการกด keystroke อย่างไรก็ตาม ผู้โจมตี จะเหมือนกันกับคุณคือมีการเข้ารหัสของข้อมูลในปัจจุบันพวกเขาใช้ SSH หรือ 3DES เป็นช่องทางในการเชื่อมต่อการสื่อสารกันกับเครื่อง computer พวกเราจะไม่สามารถจับการกดคีย์(keystrokes) ผ่านทางสายได้อีกต่อไปแทนการที่จะจับพวกเขาจากระบบที่เขาใช้หนึ่งในประโยชน์ของระบบที่มีการเข้ารหัสนั้นคือเครื่องปลายทางในที่นี้คือเครื่อง honeypot ของคุณ ถ้าหากว่ามีการจับข้อมูลบนเครื่อง honeypot ของคุณ ซึ่งข้อมูลเป็นการ decryption เราสามารถส่งข้อมูลผ่านช่องทางที่ encryption ได้ ตัว Sebek เป็นเครื่องมือที่กระทำการนี้โดยตรงตัว Sebek จะซ่อนตัวอยู่ใน kernel ซึ่งเป็น module (หรือบางทีเรียกว่า patch) ความสามารถในการจับตาความเคลื่อนไหวของเหล่า Attacker หนึ่งในกรติดตั้งตัว honeypot ตัว Sebek client จะทำงานอยู่ใน kernel ตัวข้อมูลจะถูกจับกลุ่มรวมกันโดย Sebek client โดยจะไม่เก็บไว้ในส่วนของตัว Honeypot เองเพราะว่ามันเป็นที่ซึ่งเหล่าผู้โจมตีจะสามารถเข้าไปค้นพบได้ แทนที่โดย ตัว Sebek client จะส่งข้อมูลผ่านทาง UDP เป็นการตรวจจับตัวเครื่องจักร เช่น ตัว Honetwall Gateway หรือ การ Remote login ไปยังตัวระบบต่างๆ ผู้โจมตีจะไม่สามารถเห็น และ ก็ไม่เห็น packet ของตัว sniff เช่นกัน ราวกับว่าตัว Sebek Client นั้นซ่อนตัวอยู่บน Honeypot และหลบซ่อนไม่ให้เขาเหล่านั้นเห็น เหตุการณ์หากว่าผู้โจมตี download หรือ เขาใช้เครื่องมือในการดักจับ Sniffing Sebek เองจะทำงานโดยการซ่อนตัวจากพวกเขาเหล่านั้นนี่เป็นการทำโดยปรับปรุงตัว honeypot ให้มันไม่เกิดการดักจับ packet กับการออกแบบเริ่มต้นเป็นคล้ายกับเวทมนต์ที่ได้ทำให้ไม่เห็นทั้งจำนวน และ port ของ UDP ที่เราเปิด ตัว Sebek เองได้ทำการส่งข้อมูลของเหล่าผู้โจมตีไปบนสายกับการตรวจจับโดยตัว Gateway ตั้งแต่นั้นมาตัว Honeypot เองก็ถูกควบคุมโดย Sebek ไม่มีผู้ใดที่จะสามารถจับการกดคีย์ (keystroke) บนสายได้ (หมายเหตุ: ถ้าคุณมีตัว Honeypot แต่คุณเองไม่มี Sebek ติดตั้งอยู่ หรือ ตัว Sebek คุณติดตั้งแล้วทำการ Configure ไม่ถูกต้อง) และเหล่าผู้โจมตีจะเข้ามาควบคุมระบบของคุณได้ดังนั้นเมื่อเขาสามารถที่จะดักฟัง packet ที่เข้ามาจากระบบอื่นๆ เหล่า Packet จะไม่ถูกซ่อนตัวอีกต่อไป

ตัว Sebek นี้ทำงานใน Kernel โดยมีการ Compile มาสำหรับแต่ละชนิดของ OS และ Kernel version นั้นๆ ของ Honeypot ของคุณในขณะที่ Client รุ่นอื่นๆ ที่ต่างกัน และ ระบบปฏิบัติการที่ต่างกัน พวกมันจะมีการ Configure ระบบในลักษณะที่คล้ายกัน file ข้างล่างเป็นตัวอย่าง วัตถุประสงค์ของการConfigure ตัวระบบเพื่อกำหนดข้อมูลที่ถูกรวบรวม และอย่างไรก็ตามข้อมูลจะถูกส่งไปตามสาย โดยปกติแล้ว Sebek เองจะตรวจจับทุกๆ การกระทำบนระบบอย่างไรก็ตามคุณก็ยังมีทางเลือกที่จะเลือกตรวจจับการกดคีย์เพียงอย่างเดียวก็ได้ การรวบรวมระหว่างหมายเลขอันน่าอัศจรรย์ (แล้วแต่ชนิดของ Sebek) และ ปลายทางของหมายเลข UDP port ที่ตัดสินใจว่า packet ใดบ้างที่จะซ่อนตัว

ทุกๆ Honeypot ในกลุ่มเดียวกันจะมีการแบ่งการทำงานที่ร่วมกันได้เพื่อให้ได้มาซึ่งผลประโยชน์ที่แท้จริง

```
#---- INTERFACE:
INTERFACE="eth0"
#---- DESTINATION_IP:
DESTINATION_IP="10.0.0.1"
#---- DESTINATION_MAC:
DESTINATION_MAC="FF:FF:FF:FF:FF:FF"
#---- SOURCE_PORT:
SOURCE_PORT=1101
#---- DESTINATION_PORT:
DESTINATION_PORT=0
#---- MAGIC_VAL
MAGIC_VAL=0
#---- KEYSTROKE_ONLY:
KEYSTROKE_ONLY=0
#---- TESTING:
TESTING=0
```

หนึ่งในการ Configure Sebek เองจะนำข้อมูลทั้งหมดของระบบเข้าสู่ระบบเครือข่ายมี packet ที่พวกเขาเคยใช้สร้างขึ้นมาโดยเหล่าผู้โจมตี และ ทำให้สะดวกเมื่อมีผู้โจมตี โจมตีระบบของคุณมีการเรียนรู้เกี่ยวกับ Sebek สามารถอ่านได้จาก หัวข้อ Sebek

เครื่องมือที่ใช้ในส่วนของการเก็บบันทึกผล

MySQL server

ใช้ในการจัดเก็บข้อมูลลงฐานข้อมูล MySQL เพื่อความเป็นระบบ และสามารถเข้าใช้งานได้อย่างเป็นระบบ เพื่อเชื่อมต่อข้อมูลให้โปรแกรมควบคุมสามารถมาดึงไปใช้งานได้ และรองรับการเชื่อมต่อข้อมูลในการพัฒนาในอนาคต

Sebek_server

เซเบคเซิร์ฟเวอร์ เป็นส่วนที่ทำหน้าที่รับค่าข้อมูลที่เซเบคไคลเอนต์จับได้จากเครื่องกับดัก และระบบการทำงานจะส่งค่ามาเก็บยังฐานข้อมูลที่ได้จัดเตรียมไว้ ซึ่งจะส่งผ่านมาด้วยโปรโตคอล ยูคิพี พร้อมทั้งเข้ารหัสลับไว้ โดยไม่จำเป็นจะต้องระบุไอพีปลายทาง เพื่อความปลอดภัยจะระบุเป็น MAC Address ของเครื่อง server ซึ่งเครื่องไคลเอนต์ไม่จำเป็นต้องมีการ ARP Request เพื่อหาปลายทาง ซึ่งจะช่วยลดความผิดสังเกตจากผู้บุกรุกได้ และการทำงานของ Sebek client จะทำการ by-pass ผ่าน TCP stack ก็มันจะสร้าง packet เองเลย ทำให้โปรแกรมจำพวก sniffer ที่ผู้บุกรุกอาจนำเข้ามาติดตั้งในเครื่องกับดัก ไม่สามารถตรวจพบ

Samhain (Yule)

เซมเฮนในส่วนนี้จะป็นเซอรัวิสที่ทำหน้าที่ในการรับข้อมูลจากเซมเฮนที่อยู่บนเครื่องกับดัก มาบันทึกเก็บไว้ในฐานข้อมูลที่ได้จัดเตรียมไว้

3.2.4 ส่วนการจัดการและตรวจสอบความเสียหายของเครื่องกับดัก

ในส่วนนี้เราได้สร้างโปรแกรมดูแลจัดการและควบคุมตัวระบบอันนี้เพื่อทั้งหมด เข้าไว้ในโปรแกรมเดียว Tartarus Management ซึ่งสามารถควบคุมเครื่องที่ทำหน้าที่ต่างๆทั้งหมดได้ รวมถึงความสามารถที่ใช้ในการตรวจสอบสภาพของเครื่อง Cage โดยได้สร้าง TMdaemon ซึ่งจะทำงานตลอดเวลาอยู่ที่เครื่องกับดัก คอยตรวจเช็คสภาพ และนำข้อกำหนดที่เราตั้งในตัวโปรแกรมดูแลจัดการไปเปรียบเทียบว่า ถึงจุดที่จะต้องหยุดให้บริการแล้วหรือยัง โดยในขั้นแรกนี้เราจะตรวจสอบว่าถ้าหาก Password ของ Administrator ถูกเปลี่ยนเราจะหยุดการให้บริการเครื่องกับดักเครื่องนั้นๆ โดยค่าที่ตั้งนี้หากนำไปใช้กับระบบอื่นๆก็สามารถเปลี่ยนแปลงได้เพื่อความเหมาะสมกับระบบขององค์กรนั้นๆ เราจำเป็นต้องให้มีการตรวจสอบไม่ให้เครื่อง Cage อ่อนแอจนเกินไป เพื่อไม่ให้ผู้บุกรุกใช้เครื่อง Cage เป็นฐานที่มั่นในการโจมตีระบบอื่นๆ ซึ่งจะก่อให้เกิดความเสียหายต่อระบบจริงขององค์กร

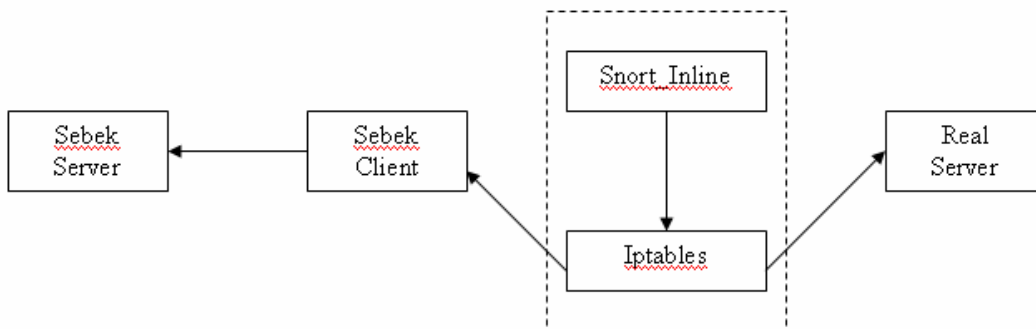
โดยเราสามารถเรียกดูสภาพของเครื่องCageได้ เช่นมีไฟล์ใดที่ถูกเปลี่ยน และสามารถหยุดการทำงานพร้อมทั้งสร้างCageตัวใหม่ได้โดยผ่านโปรแกรม TM และเครื่องกับดักที่หยุดให้บริการก็จะถูกเก็บไว้เพื่อใช้ในการศึกษาต่อไป

3.2.5 ส่วนการควบคุมและแสดงผลการบันทึกข้อมูลต่างๆของระบบ

การดูแลและการจัดการระบบอันนี้เพื่อต้งทั้งหมดจะกระทำผ่านโปรแกรมTM (Tartarus Management) เพื่ออำนวยความสะดวกให้แก่ผู้ดูแลระบบ โดยโปรแกรมมีความสามารถควบคุมเครื่องมือในส่วนต่างๆของระบบได้ทั้งหมด เช่น การสั่งหยุดการให้บริการเครื่องกับดัก การสร้างเครื่องกับดักใหม่ หรือการเพิ่มหรือถอดถอนกฎที่ใช้เป็นต้น

และได้จัดเตรียมช่องทางปลอดภัยในการให้ ผู้ดูแลระบบเข้ามาควบคุมระบบอันนี้เพื่อต้งได้ โดยผ่าน IPsec ที่จะมีการ Authentication และการ Encryption ข้อมูลที่มีการส่งถึงกัน ดังนั้นผู้ดูแลระบบไม่จำเป็นต้องไปทำงานหน้าเครื่องHoneywall แต่สามารถควบคุมระบบผ่านทางเครือข่ายได้อย่างปลอดภัย ช่วยให้สามารถแก้ปัญหาที่เกิดขึ้นได้ทันที หากเกิดเหตุการณ์ผิดปกติเกิดขึ้น

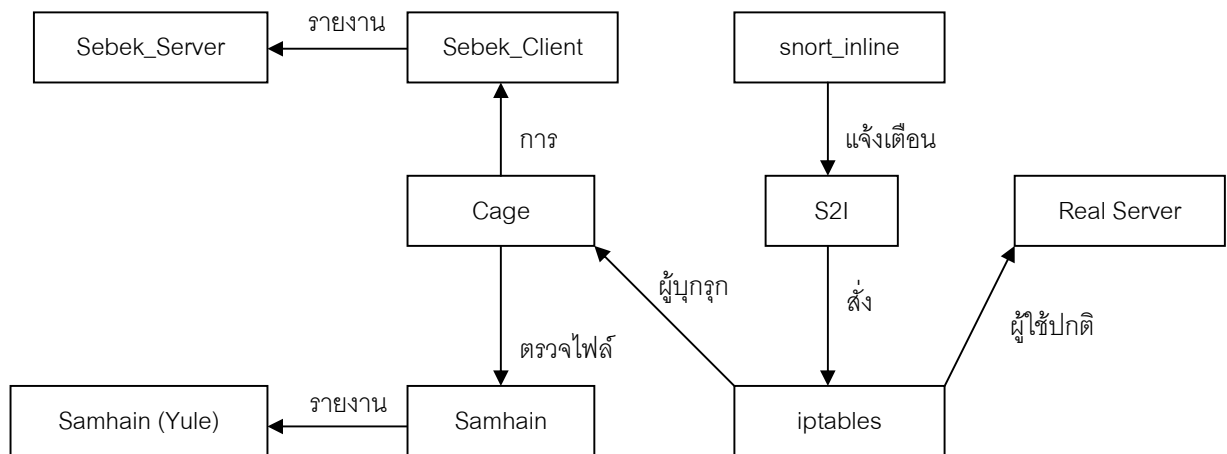
3.3 การทำงานร่วมกันของระบบ



รูปที่ 3.3 แสดงการเชื่อมต่อระหว่างส่วนต่าง ๆ เพื่อทำการจำแนกผู้ใช้งานปกติกับผู้บุกรุกแบบเก่า

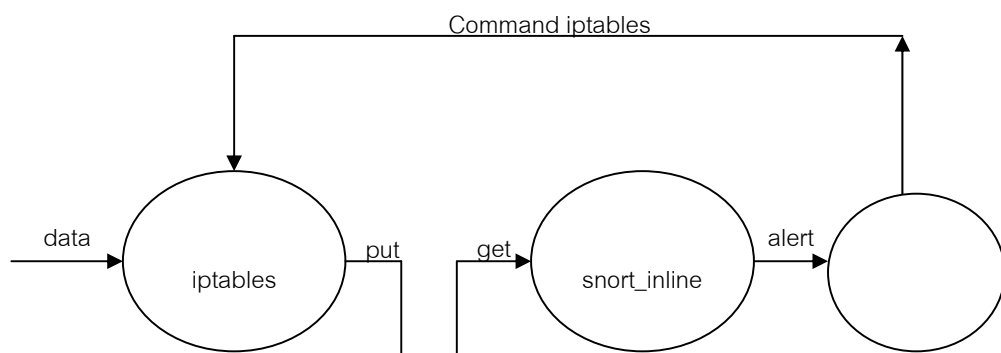
จากรูปเป็นลักษณะการทำงานของโปรแกรมจำแนกและบันทึกพฤติกรรมผู้บุกรุกที่ได้สร้างขึ้นในปีการศึกษาที่ผ่านมา ซึ่งการทำงานจะเริ่มจาก snort_inline ทำการตรวจสอบและแยกแยะระหว่างผู้บุกรุกกับผู้ใช้งานทั่วไปจากนั้นจะติดต่อไปยังโปรแกรม iptables เพื่อให้จัดเส้นทางการใช้งานระบบเครือข่ายอย่างเหมาะสมตามการจำแนกที่ snort_inline โดยจัดการเส้นทางให้ผู้บุกรุกผ่านไปยังกับดักที่มี sebek client คอยดักจับและส่งการกระทำต่างๆ ของผู้บุกรุกไปให้ sebek server จัดเก็บลง

ฐานข้อมูลอยู่ตลอดเวลา ส่วนเส้นทางของผู้ใช้งานปกติให้ผ่านไปยังเซิร์ฟเวอร์จริง แต่ในปีการศึกษานี้ได้มีการปรับแต่งและเพิ่มบางส่วนเพื่อให้การทำงานของโปรแกรมมีประสิทธิภาพมากขึ้นโดยได้ทำการเพิ่มโปรแกรม samhain ซึ่งเป็นส่วนที่จะทำงานกับเครื่องกับดักโดยโปรแกรม samhain จะคอยตรวจสอบความถูกต้องของไฟล์สำคัญในระบบของเครื่องกับดัก ซึ่งเราสามารถกำหนดได้ว่าจะให้ทำการตรวจสอบไฟล์ใด ซึ่งการตรวจสอบความถูกต้องของไฟล์บางไฟล์นั้นสามารถใช้เพื่อการตัดสินใจในการเปลี่ยนเครื่องกับดักเนื่องจากสภาพของเครื่องกับดักมีความอ่อนแอจนอาจที่จะไม่สามารถควบคุมพฤติกรรมของผู้บุกรุกให้อยู่ในขอบเขตที่ควรจะเป็น (ไม่สามารถใช้เครื่องกับดักเป็นฐานที่มั่นในการโจมตีเครื่องอื่นๆ ทั้งในระบบและนอกระบบ) ได้ เนื่องจากโปรแกรมมีการเก็บด้วยกว่าไฟล์ถูกเปลี่ยนแปลงไปอย่างไร เช่นถูกเปลี่ยนเจ้าของไฟล์จาก root ไปเป็นบุคคลอื่น เป็นต้น



รูปที่ 3.4 แสดงการเชื่อมต่อระหว่างส่วนต่าง ๆ เพื่อทำการจำแนกผู้ใช้งานปกติกับผู้บุกรุกแบบใหม่

ซึ่งในส่วนของการติดต่อกันระหว่าง snort_inline และ iptables นั้นเป็นโปรแกรมที่ทำงานคนละโหมดกันโดยโปรแกรม snort_inline นั้นจะทำงานในยูสเซอร์โหมดแต่โปรแกรม iptables นั้นทำการในเคอร์เนลโหมด ดังนั้นโปรแกรมทั้ง 2 โปรแกรมนี้จึงไม่สามารถที่จะติดต่อกันเองได้ ดังนั้นจึงต้องมีโปรแกรมสื่อกลางที่จะคอยแปลงคำแจ้งเตือนที่มาจาก snort_inline ไปเป็นคำสั่งที่โปรแกรม iptables นั้นเข้าใจ เพื่อให้การทำงานของระบบเป็นไปได้อย่างมีประสิทธิภาพ ซึ่งในส่วนนี้ยังคงใช้โปรแกรมสื่อกลางดังกล่าวจะยังคงใช้โปรแกรม s2i ที่ได้มีการพัฒนาขึ้นในปีที่ผ่านมาซึ่งลักษณะการทำงานเป็นไปตามรูปด้านล่าง



รูปที่ 3.5 แสดงการทำงานของส่วนจำแนกและกำหนดทิศทางการเชื่อมต่อ

3.4 สรุปการทำงานโดยรวมของระบบ

ในการทำงานโดยรวมของระบบอันนี้เพื่อที่จะเริ่ม จากส่วนการจำแนกและคัดแยกระหว่างผู้ใช้งานทั่วไปกับผู้บุกรุก จะเป็นหน้าที่การทำงานของตัวเกตเวย์ เรียกกันว่าฮันนีวอลล์ (Honeywall) ซึ่งเป็นส่วนที่ใช้เพื่อเป็นทางผ่านของขึ้นข้อมูลทุกชิ้นที่จะเข้ามาในระบบเครือข่าย ในลักษณะการติดตั้งตัวเกตเวย์ไว้ที่ส่วนที่เสมือนประตูทางเข้าออกนี้ เป็นส่วนที่สามารถใช้ประโยชน์ของระบบตรวจจับผู้บุกรุก (snort) เข้าไปเพื่อให้การวิเคราะห์และจำแนกผู้บุกรุกนั้นมีประสิทธิภาพสูงสุด แต่ในส่วนการทำงานเพียงลำพังของระบบตรวจจับผู้บุกรุก นั้นไม่เพียงพอที่จะจัดการให้ผู้บุกรุกเข้าสู่ระบบกับดักที่จัดเตรียมไว้ได้ เนื่องจากระบบตรวจจับผู้บุกรุกนั้นไม่มีความสามารถเพียงพอในการจัดการด้านช่องทางการเชื่อมต่อ ซึ่งความสามารถนี้เป็นความสามารถของไฟร์วอลล์ ที่มีชื่อเรียกว่า IPtables เพื่อเป็นตัวที่เพิ่มขึ้นมาเพื่อเป็นตัวจัดการ การเชื่อมต่อสื่อสารเพื่อให้เป็นไปตามความต้องการของผู้พัฒนา แต่ในการทำงานจริงนั้น ทั้งสองโปรแกรมนี้ไม่สามารถพูดคุยเพื่อควบคุมกันได้ เพราะความสามารถที่มีนั้นเป็นแค่เพียง ไฟร์วอลล์นั้นส่งค่าของขึ้นข้อมูลไปยังส่วนที่โปรแกรมในส่วนยูเซอร์โหมด (user mode) ก็คือตัวระบบตรวจจับผู้บุกรุก (snort) เนื่องจากว่าไฟร์วอลล์เองเป็นโปรแกรมที่ทำงานในส่วนของเคอร์เนลโหมด (kernel mode) ดังนั้นจึงได้มีการใช้งานตัวกลางที่จะสามารถสื่อสารระหว่างระบบตรวจจับผู้บุกรุกกับไฟร์วอลล์เป็นตัวที่ใช้การอ่านค่าจากการแจ้งเตือนจากระบบตรวจจับผู้บุกรุก และอ่านค่าที่ได้มานั้นแปลงเป็นคำสั่งของไอพีเทเบิล อย่างเหมาะสมเพื่อสั่งให้จัดส่งขึ้นข้อมูลไปยังปลายทางที่ถูกต้องS2I(Snort command to Iptables)หลังจากนั้นผู้บุกรุกจะถูกเปลี่ยนแปลงการทำงานให้ไปมีผลต่อเครื่องกับดักแทน โดยเครื่องที่ทำหน้าที่เป็นกับดัก(cage)ทางผู้พัฒนาได้พัฒนาให้มีความปลอดภัยต่อองค์กร นั่นคือผู้บุกรุกจะไม่สามารถใช้เครื่องกับดักไปเป็นฐานโจมตีเครื่องอื่นๆ และพัฒนาให้มีความแนบเนียนเสมือนว่าเป็นเครื่องให้บริการจริงที่เป็นเป้าหมายของผู้บุกรุก โดยจะตรวจสอบไม่พบความผิดปกติของค่าต่างๆที่ระบุว่าเป็นเครื่องกับดัก เช่น ไอพีแอดเดรส แมค

แอดเดรส หรือไม่พบเซิร์ฟเวอร์ที่ทำให้ทราบว่าได้มีการเฝ้าดูการกระทำต่างๆ เช่น ทุกๆ keystroke ที่ผู้บุกรุกส่งงานเข้ามา แม้ผู้บุกรุกจะใช้คำสั่งผ่านการเข้ารหัสก็ตาม หรือแม้แต่การนำเข้าไฟล์หรือส่งออก packet ตัวเครื่องกับดักก็จะส่งข้อมูลทุกอย่างไปเก็บไว้ยังเครื่องแม่ข่ายฐานข้อมูล (Log server) ซึ่งการส่งข้อมูลออกไปจะส่งไปแบบเข้ารหัสและไร้ตัวตน โดยที่ผู้บุกรุกไม่ทราบว่าได้มีการส่งข้อมูลออกไป ด้วยเทคนิคของ Sebek ซึ่งทำงานในแบบ kernel-base นั่นคือแม้ว่าผู้บุกรุกจะนำโปรแกรมดักจับแพ็คเก็ตมาติดตั้งก็จะไม่พบว่าเครื่องกับดักได้ส่งข้อมูลออกไป นอกจากนี้เครื่องกับดักยังได้ติดตั้งเครื่องมือที่ใช้ในการตรวจสอบความเปลี่ยนแปลง และตรวจสอบสภาพความเสียหายเอาไว้ และทำการตรวจสอบอยู่ตลอดเวลา เมื่อถึงจุดที่มีความเสียหายตามที่กำหนดเครื่องกับดักก็จะหยุดให้บริการโดยอัตโนมัติ

ในส่วนของการจัดการและเรียกดูข้อมูลทางผู้พัฒนาได้สร้างชุดโปรแกรม TM (Tartarus Management) ที่สามารถติดต่อกับงานร่วมกับส่วนต่างๆ เพื่อสามารถจัดการและบริหารระบบได้โดยใช้เพียงโปรแกรมเดียว และยังสามารถเรียกดูข้อมูลทั้งหมดที่มีการบันทึกผ่านโปรแกรมนี้ได้ทันที โดยโปรแกรมนี้อย่างยังสามารถส่งการผ่านเครือข่ายได้โดยปลอดภัยโดยใช้ IPsec (IP security) ซึ่งจะเพิ่มความสะดวก และรวดเร็วในการเข้าแก้ปัญหาที่อาจเกิดขึ้นในระบบจริง เพราะผู้ดูแลระบบไม่จำเป็นต้องเข้ามาควบคุมหน้าเครื่อง Honeywall แต่สามารถควบคุมผ่านทางเครือข่ายได้เลย

3.5 ข้อจำกัดและข้อด้อยของฮันนี่พ็อตเดิม

3.5.1 ความปลอดภัยของเครื่องกับดัก (Cage) โดยผู้บุกรุกอาจใช้เครื่องกับดักเป็นฐานที่มั่นในการไปโจมตีเครื่องอื่นๆ

3.5.2 ความแม่นยำของเครื่องกับดัก (Cage) ผู้บุกรุกอาจตรวจสอบพบว่าข้อมูลและค่าประจำเครื่อง ไม่ได้เป็นเครื่องเดียวกับเครื่องให้บริการจริง ทำให้ผู้บุกรุกรู้ตัวและไม่กระทำการใดๆ

3.5.3 ความยุ่งยากในการนำมาใช้งานจริง เนื่องจากระบบฮันนี่พ็อตต้องการทำงานของเครื่องที่ทำหน้าที่ต่างๆ และแต่ละเครื่องต้องมีการใช้งานหลายเครื่องมือ ดังนั้นการบริหารจัดการและการดูแลสภาพของระบบจึงมีความซับซ้อนยุ่งยาก

3.5.4 การเปลี่ยนแปลงต่างๆที่เกิดจากผู้บุกรุก เมื่อผู้บุกรุกมีการใช้งานเครื่องมือต่างๆ แล้วไปเปลี่ยนแปลงไฟล์ใดๆ ทางฮันนี่พ็อตไม่สามารถติดตามได้ ตัวอย่างเช่น ทางระบบบันทึกได้ว่าผู้บุกรุกได้นำ script หรือเครื่องมือใดๆเข้ามา และทำการเรียกใช้งาน แต่ไม่ทราบว่าเครื่องมือเหล่านี้ไปเปลี่ยนแปลงไฟล์หรือกระทำการใดๆบ้าง

3.6 รูปแบบระบบอันนี้พ็อตที่เราได้พัฒนาขึ้นมาใหม่

ระบบอันนี้พ็อตที่ได้พัฒนาแล้วจะช่วยลดจุดด้อยที่มีอยู่และเพิ่มประสิทธิภาพของการทำงานเดิม ดังนี้

3.6.1 เครื่องกับดักมีความปลอดภัย จะไม่สามารถถูกใช้ไปโจมตีเครื่องอื่นๆ ได้ เพราะจะถูกระบบการใช้งานทันที โดยอัตโนมัติเมื่อถึงจุดที่เครื่องมีความอ่อนแอ ตามที่ผู้ดูแลระบบเห็นสมควร ซึ่งแต่เดิมผู้ดูแลระบบจะต้องหมั่นมาตรวจสอบสภาพเครื่องกับดักเอง

3.6.2 เครื่องกับดักมีความแม่นยำ โดยเมื่อผู้บุกรุกตรวจสอบเครื่องว่าเป็นเครื่องใด เช่น ใช้คำสั่งตรวจสอบไอพีแอดเดรส หรือตรวจสอบแมคแอดเดรส

3.6.3 ปัญหาความยุ่งยากด้านการควบคุมและดูแลระบบอันนี้เเนตถูกทำให้ลดน้อยลง โดยการมีโปรแกรมซึ่งเป็นศูนย์กลางควบคุมระบบทั้งหมด ซึ่งได้มีคำสั่งของเครื่องมือและองค์ประกอบที่จำเป็นอย่างครบถ้วน ผู้ใช้จึงไม่จำเป็นต้องทราบ คำสั่งต่างๆ ที่ใช้ในการควบคุมและสั่งงาน รวมทั้งการเรียกดูข้อมูลที่ได้จัดเก็บไว้ โดยโปรแกรมนี้จัดทำในแบบ Graphic User Interface จึงยังง่ายในการใช้งานของระบบอันนี้พ็อต

3.6.4 เพื่อติดตามการเปลี่ยนแปลงของระบบ จึงได้จัดหาเครื่องมือที่จะช่วยตรวจสอบการเปลี่ยนแปลงของระบบได้ ว่าไฟล์ใดถูกเปลี่ยนแปลง ไฟล์ใดถูกลบทำลาย จึงเก็บรายละเอียดการกระทำของผู้บุกรุกได้ละเอียดยิ่งขึ้น

3.6.5 สามารถทำการป้องกันและดักจับโค้ดหนอนที่แพร่กระจายในเครือข่ายได้ และสามารถดูพฤติกรรมต่อได้ ว่าโค้ดหนอนนั้นๆมีการกระทำใดบ้าง ไปแก้ไขไฟล์ใดบ้าง

3.6.6 เพิ่มขีดความสามารถในการรองรับจำนวนเครื่องกับดักให้มีมากและควบคุมได้ตามเท่าที่ต้องการ สามารถสร้างและเปลี่ยนเครื่องกับดักได้ทันที ที่ต้องการ

3.6.7 การเก็บรวบรวมข้อมูลทุกอย่างจัดทำขึ้นใหม่ในรูปแบบฐานข้อมูล ซึ่งจะช่วยให้การจัดการข้อมูลได้มีอย่างเป็นระบบมากขึ้น

3.7 ข้อควรระวังในการใช้งานระบบอัตโนมัติ

ในการใช้งานระบบอัตโนมัติ การกำหนดค่าความปลอดภัยและจุดที่ถือว่าจำเป็นต้องหยุดการใช้เครื่องกับค่านั้นๆแล้ว เพื่อความคลอบคลุมและยืดหยุ่น เราได้ออกแบบให้ผู้ดูแลระบบสามารถกำหนดได้เองตามความเหมาะสม โดยได้จัดเตรียมกฎเกณฑ์ต่างๆไว้ให้เลือกใช้ แต่ไม่ว่าอย่างไรถ้าหากมีการกำหนดที่ไม่เหมาะสม หรือจัดวางตำแหน่งของเครื่องมือต่างๆอย่างไม่เหมาะสม ก็จะทำให้เครื่องกับคักอาจถูกใช้เป็นฐาน ในการโจมตีเครื่องอื่นๆได้